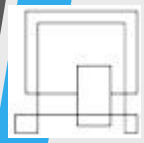
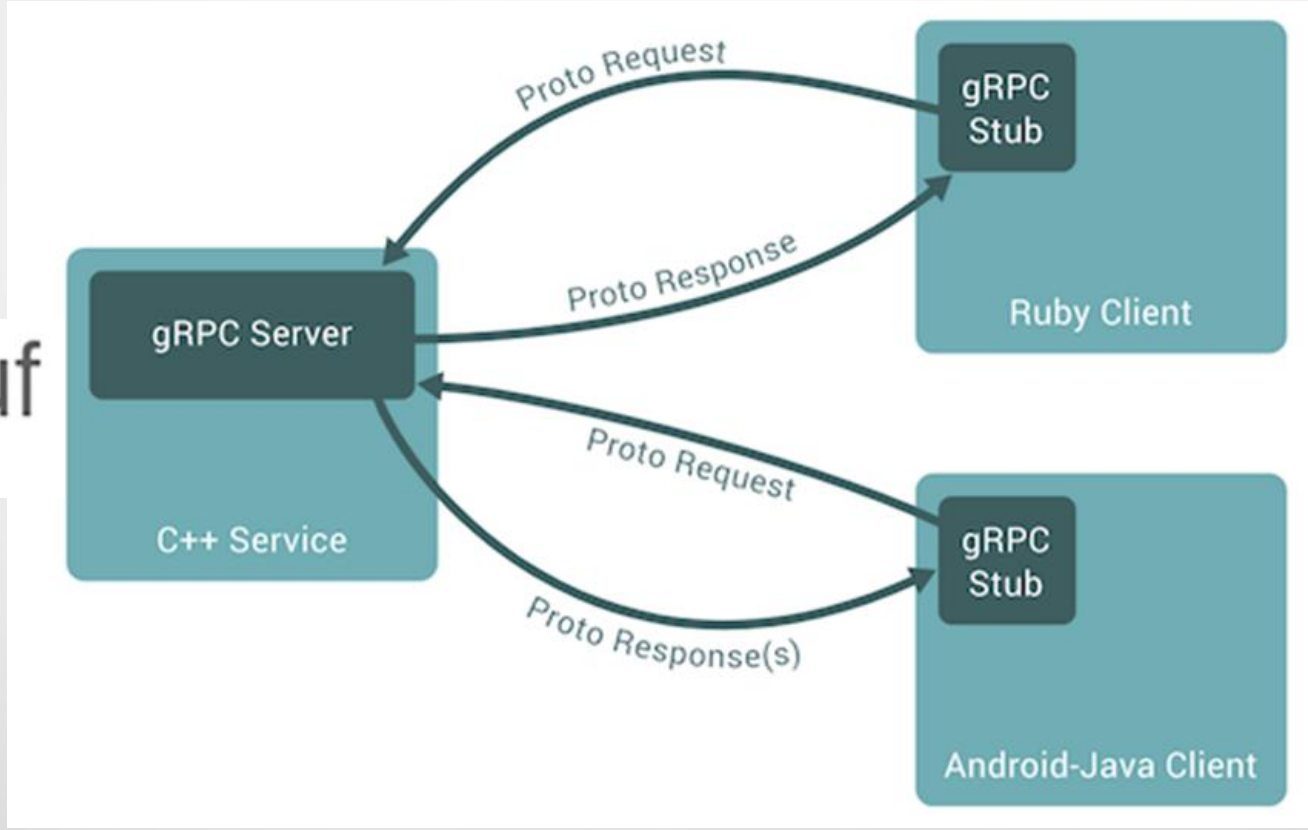
VIA University College



Programming 3

Autumn 2026

Remote Procedure Call - gRPC



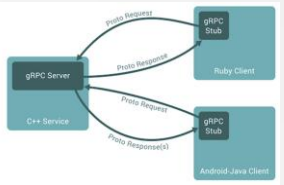
protobuf
Protocol Buffers

Learning Objectives

- By the end of this session, you should be able to:
  - ✓ Explain gRPC
  - ✓ Explain Protocol Buffers
  - ✓ implement a client-server application using the Java gRPC API

What is RPC?

- Remote Procedure Call (RPC) is a protocol that one program can use to request a service from a program located in another computer on a network without having to understand the network's details.



What is gRPC?

- RPC stack developed and used by Google
- Open sourced in Feb 2015
- Platform independent & High performance
- Based on "protocol buffers" ("protobuf") and HTTP/2
- good choice if you want efficient, robust, well-designed/implemented/tested, cross-platform RPC!

Usually, gRPC is used with protobuf

Open source Remote Procedure Call framework that can run in any environment and supports multiple programming language

Why should we use gRPC?

- Performance
  - Communicates over HTTP/2 transport protocol
  - Binary encodings via protocol buffers
  - No more parsing text
  - Compression
  - Duplex Streaming over a single TCP connection
- Remote Procedure Calls
- Polyglot

Disadvantages of gRPC?

- Drastic service definition changes are a complicated development process.
- May not be the most suitable protocol for external-facing interfaces as most of the external consumers are new about gRPC and REST/HTTP.
- The gRPC ecosystem is still small compared to the conventional REST/HTTP protocol *(but recently getting more recognition)

gRPC in the Real World



What is protocol buffers?

- Binary serialization protocol designed and used by Google
  - (JSON variant added later, but doesn't apply to gRPC)
- Open sourced in July 2008
- Platform independent
- High performance
  - Low CPU to process
  - Low bandwidth to transmit
- Version tolerant

Protocol Buffer and Protocol Files

- Protocol Buffers serves both as Interface Definition language (IDL) and message interchange format for serializing structured data
- The protofiles are records which contain name-value pairs.
- Each message defined must contain a unique numbered field and each field has a name and a type.
- Types can be numbers, booleans, strings, raw bytes, or other protocol buffer messages.

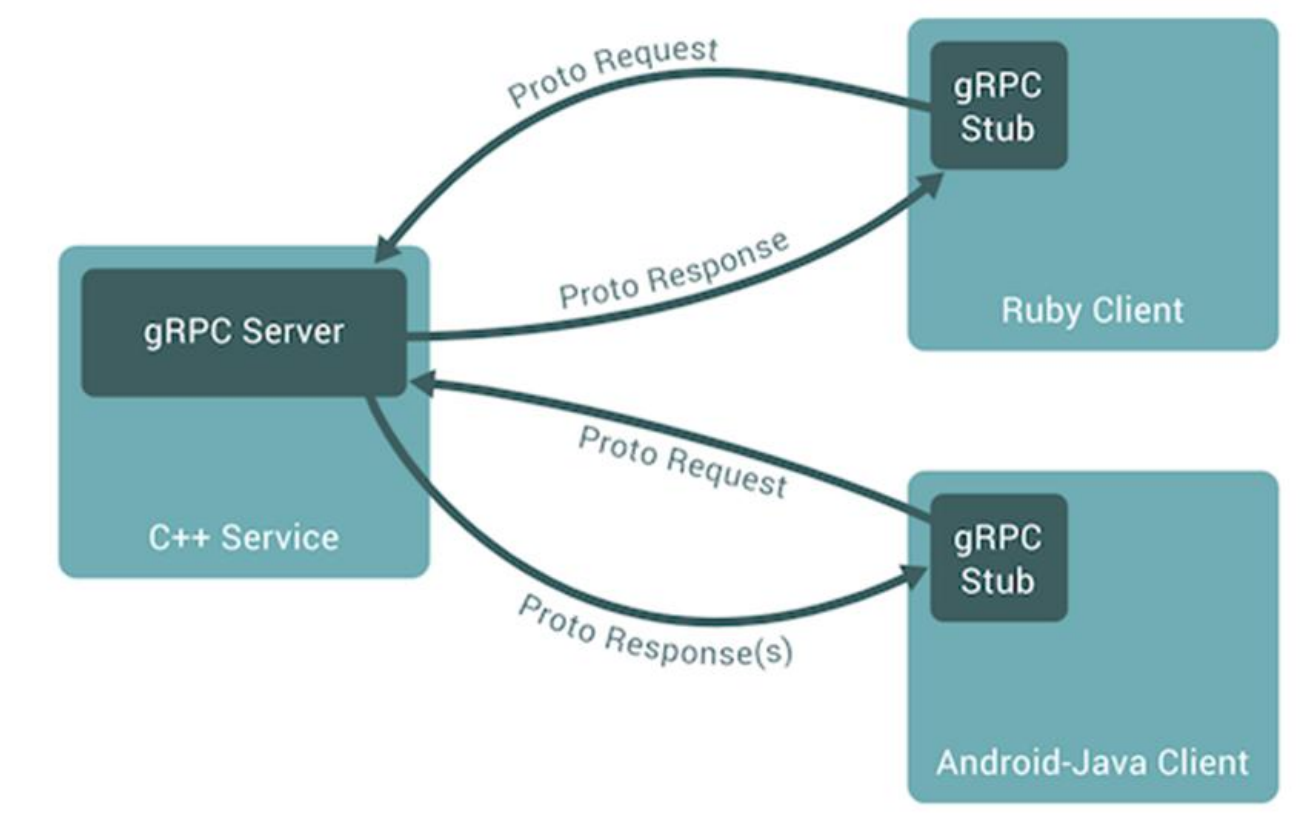
language-neutral, ***platform-neutral***

method for serializing & deserializing structured data between 2 services

Protocol Buffers – Scalar Types

Java Type	Proto Type
int	int32
long	int64
boolean	bool
double	double
float	float
String	string
byte[]	bytes

gRPC Client-Server Architecture



gRPC stub is an object that exposes service interfaces

Service Definition

- The gRPC service definition is a set of operations that need to be performed on the defined message type

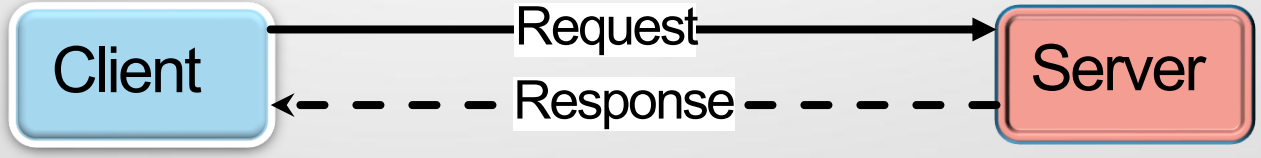
1. Unary RPC



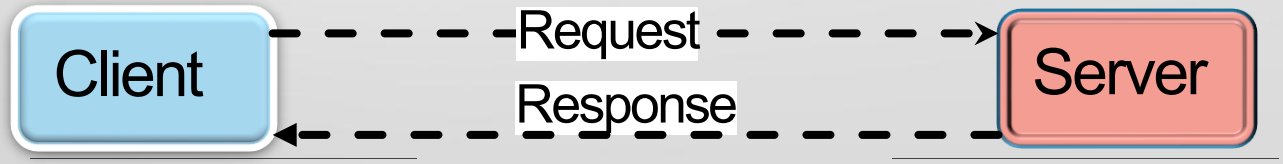
2. Client streaming RPC



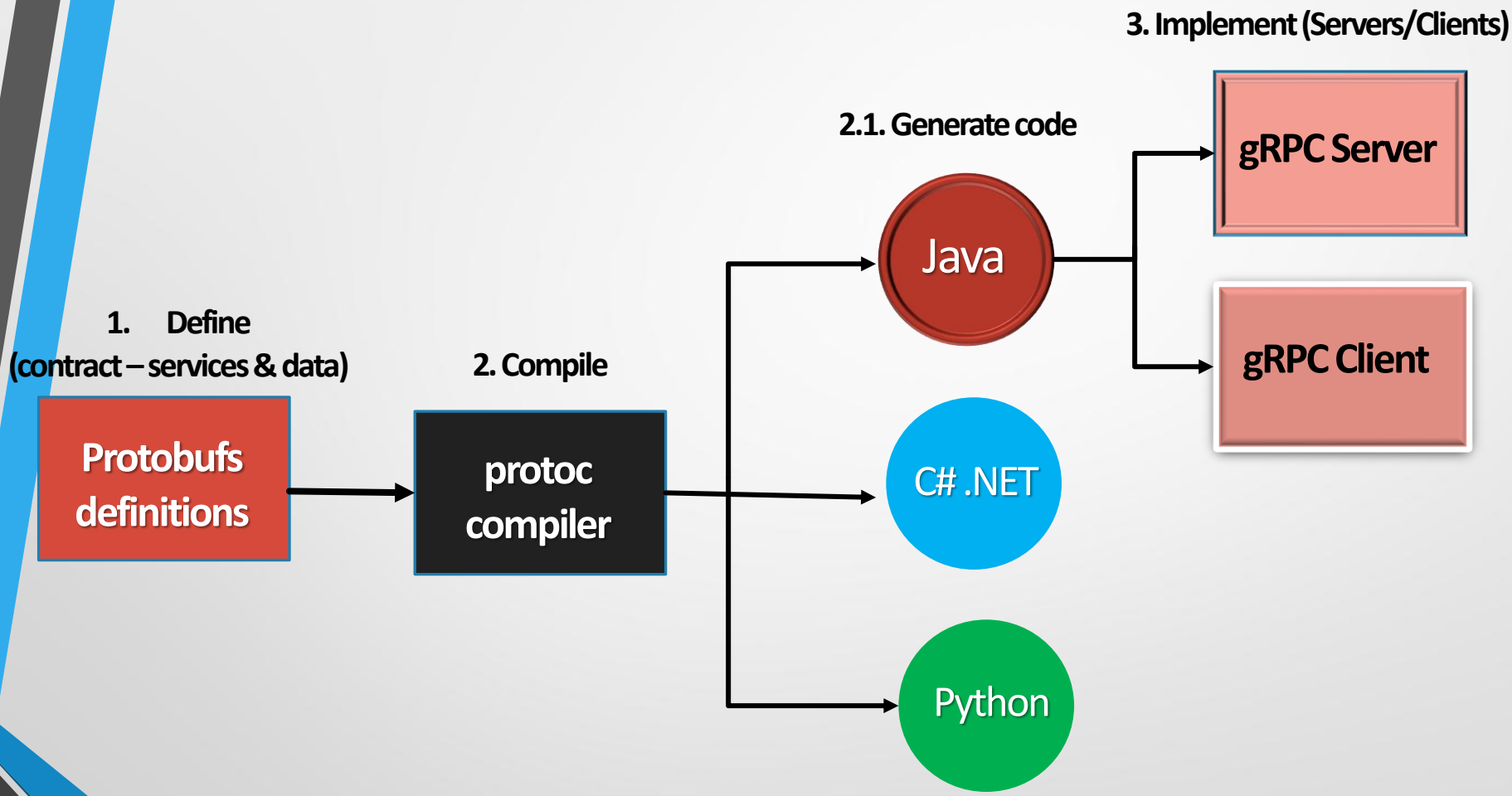
3. Server streaming RPC



4. Bi-directional streaming RPC



gRPC Workflow



Define once, generates well-structured code for all supported language!

Protocol Buffers - Example

gRPC uses protobuf to define service contracts

Like JSON, but smaller, faster, and simpler

```
syntax = "proto3";

option java_multiple_files = true;
option java_package = "via.pro3.slaughterhouse";

package slaughterhouse;

service SlaughterHouse {
  rpc totalProductWeight (stream AnimalPart) returns (Product) {}
}
message AnimalPart {
  string animal_id = 1;
  string part_id = 2;
  string name = 3;
  double weight = 4;
}
message Product {
  int32 parts = 1;
  double weight = 2;
}
```

gRPC + Protobuf

Generates:

Models(all the messages) & Java files (service base interface and stubs)

```
syntax = "proto3";

option java_multiple_files = true;
option java_package = "via.pro3.slaughterhouse";

package slaughterhouse;

service SlaughterHouse {
  rpc totalProductWeight (stream AnimalPart) returns (Product) {}
}
message AnimalPart {
  string animal_id = 1;
  string part_id = 2;
  string name = 3;
  double weight = 4;
}
message Product {
  int32 parts = 1;
  double weight = 2;
}
```

Generated SlaughterHouseImplBase

```
public static abstract class SlaughterHouseImplBase implements io.grpc.BindableService {

  public io.grpc.stub.StreamObserver<via.sdj3.slaughterhouse.AnimalPart> totalProductWeight(
      io.grpc.stub.StreamObserver<via.sdj3.slaughterhouse.Product> responseObserver) {
    return io.grpc.stub.ServerCalls.asyncUnimplementedStreamingCall(getTotalProductWeightMethod(), responseObserver);
  }
…
```

Live Coding/Hands-on/Demo

